Kleinberg - Tardos Book: Chapter 8 - Exercise 7

Berk Bubuş - 21945939
Ahmet Uman - 2210356129

Let's look at the question

Since the 3-Dimensional Matching Problem is NP-complete, it is natural to expect that the corresponding 4-Dimensional Matching Problem is at least as hard. Let us define 4-Dimensional Matching as follows. Given sets W, X, Y, and Z, each of size n, and a collection C of ordered 4-tuples of the form (wi, xj, yk, z), do there exist n 4-tuples from C so that no two have an element in common?

Prove that 4-Dimensional Matching is NP-complete.

We have some question marks

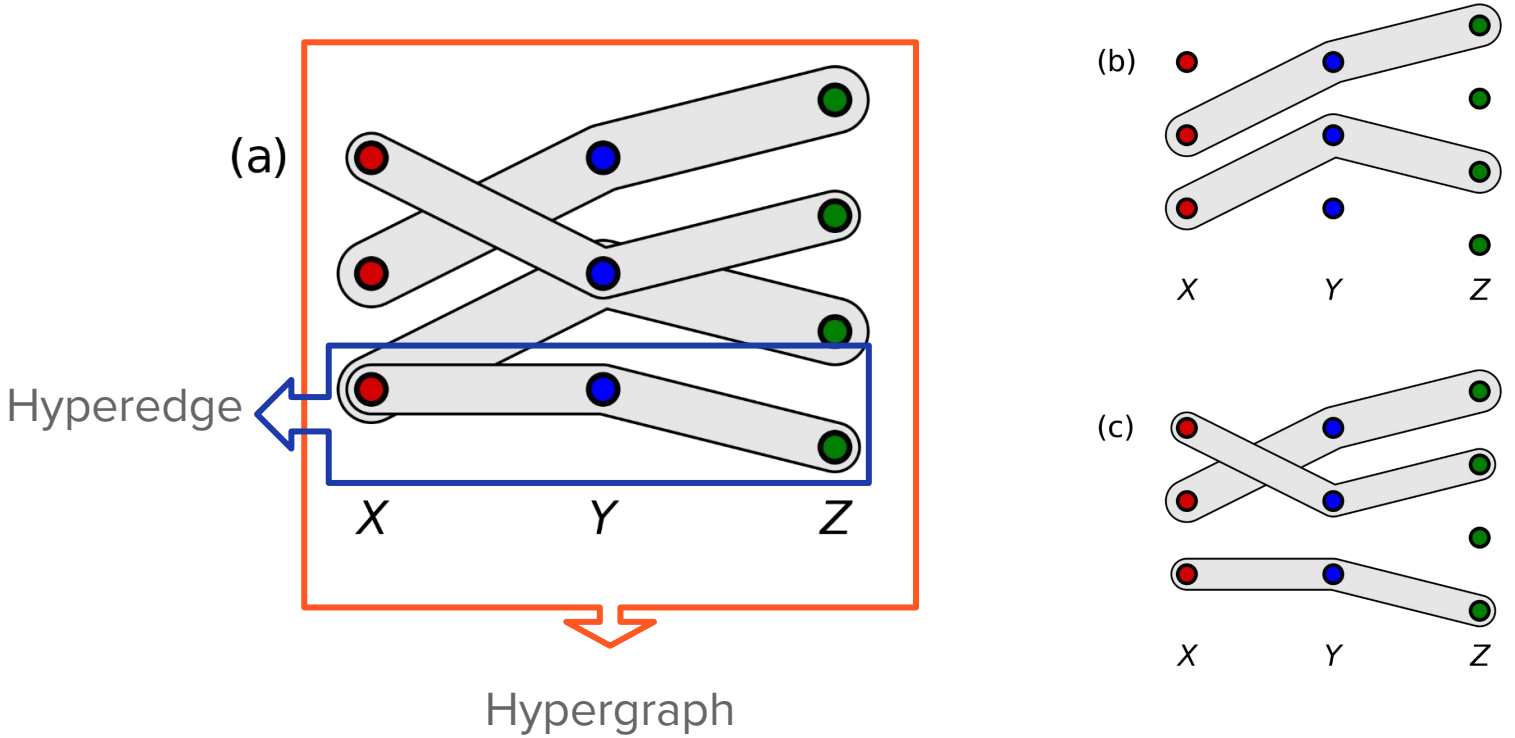
What is 3-Dimensional Matching Problem?

What is 4-Dimensional Matching Problem?

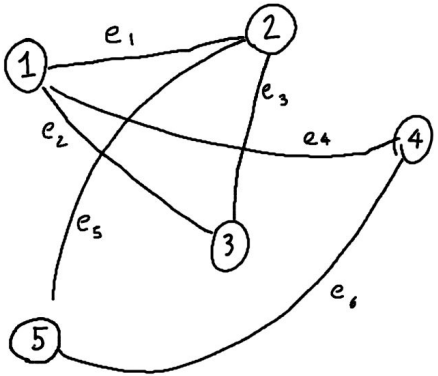
How is it NP-Complete?

How to prove?

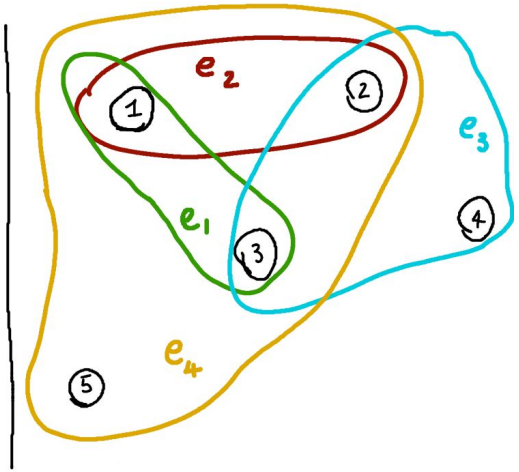
3-Dimensional Matching Problem



Hypergraphs



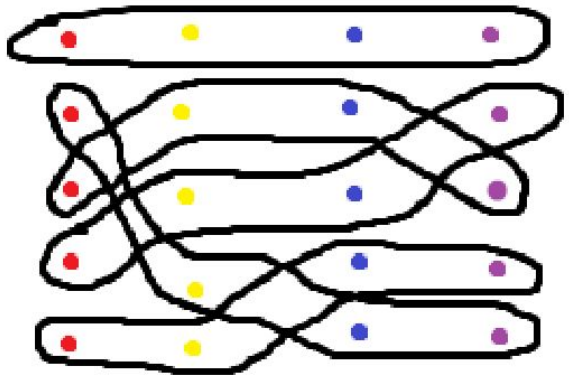
Graph



Hypergraph

4-Dimensional Matching Problem

Actually, it is similar to 3-DM but 4.



3-DM and NP-Completeness

? 3-DM is in NP

? There exists a polynomial-time reduction from a known NP-Complete problem to 3-DM

3-DM is in NP

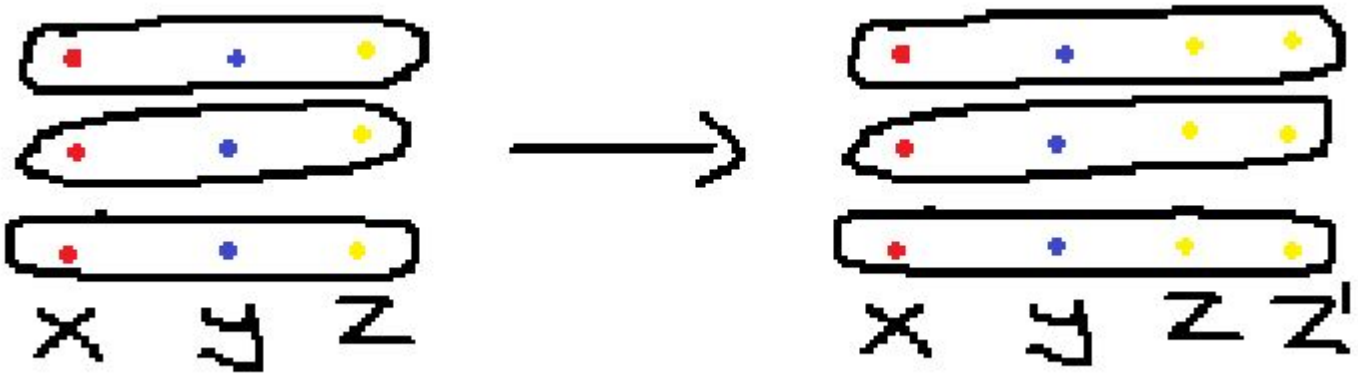
Imagine we are given a solution to a 3-DM instance, which is a collection of triples (one object each from X, Y, and Z). Verifying this solution is simple:

- Check if each triple contains one element from each set (X, Y, Z).
- Ensure no element appears in more than one triple.

This verification process can be done in polynomial time (proportional to the number of triples) by iterating through the triples and checking elements against the sets.

4-DM in NP? (3-DM to 4-DM)

For this reduction we simply ***pad*** the last position, to make the 3-tuples into 4-tuples. This can be accomplished by connecting the third position to the fourth, taking into account the sets available for selection and the set upon which the fourth set is based.



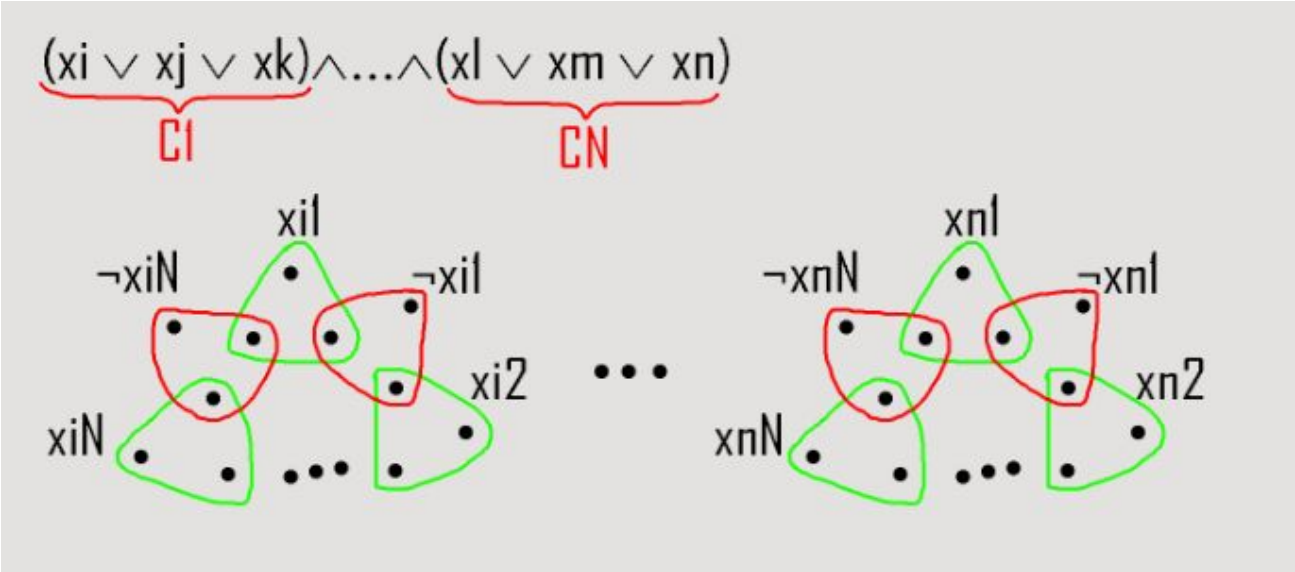
Reduction from 3-SAT to 3-DM

We have 3 parts named gadgets to reduct 3-SAT to 3-DM.

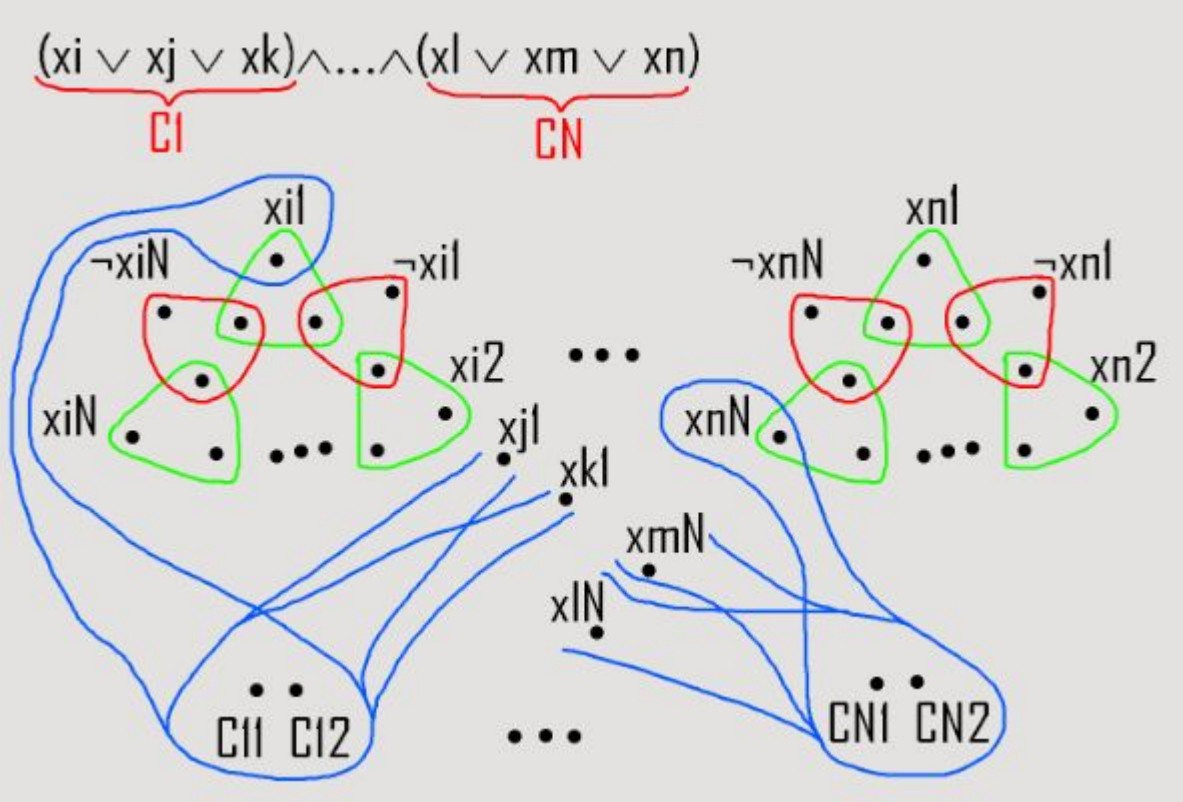
- Variable Gadget
- Clause Gadget
- Garbage Gadget

Variable Gadget

We have a gadget for each variable in clauses. For each variable, we have itself and its negation, occurring N (the clause count) times. And they are connected with nodes which help us to pick triples for 3-DM algorithm.



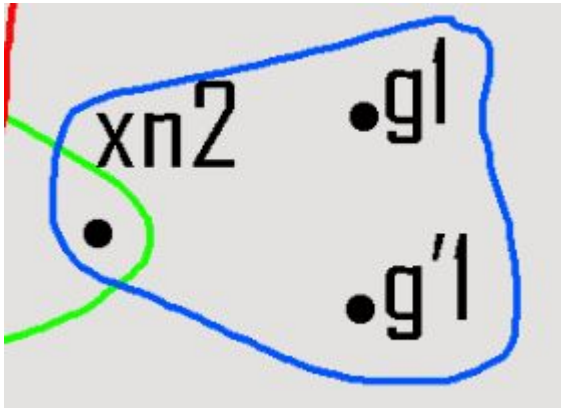
Clause Gadget



We have 2 clause point for each clause. we draw 3 hyperedges which contains clause points and one of its variables.

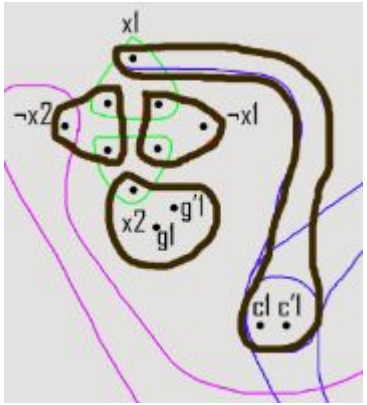
Garbage Gadget

After matching clauses, we still have free variable points. We create new g and g’ points and match them to clear free variables.



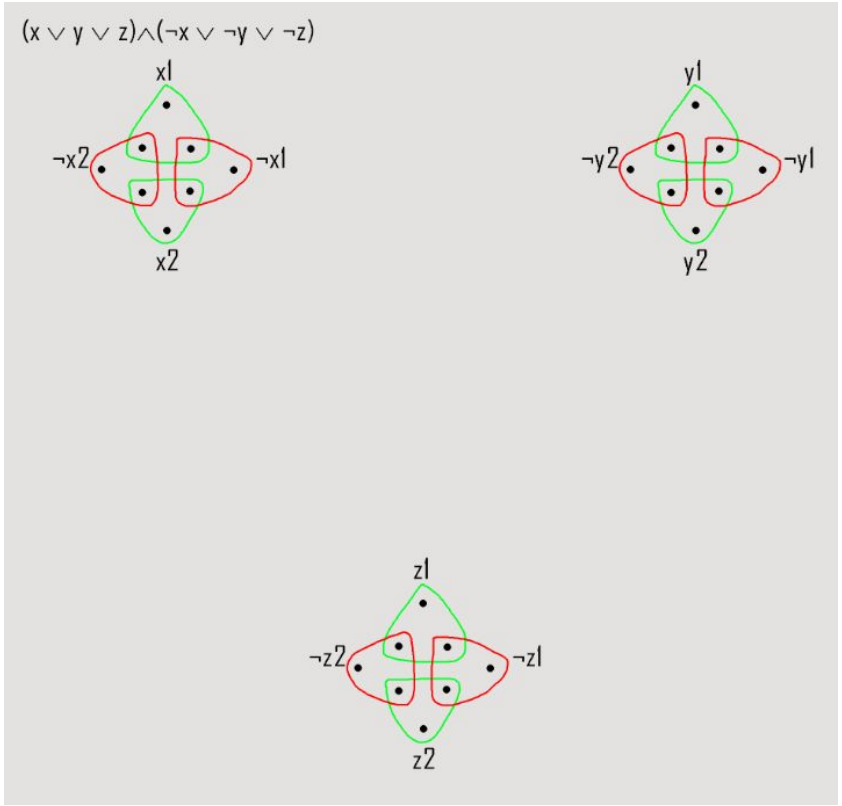
How We Choose

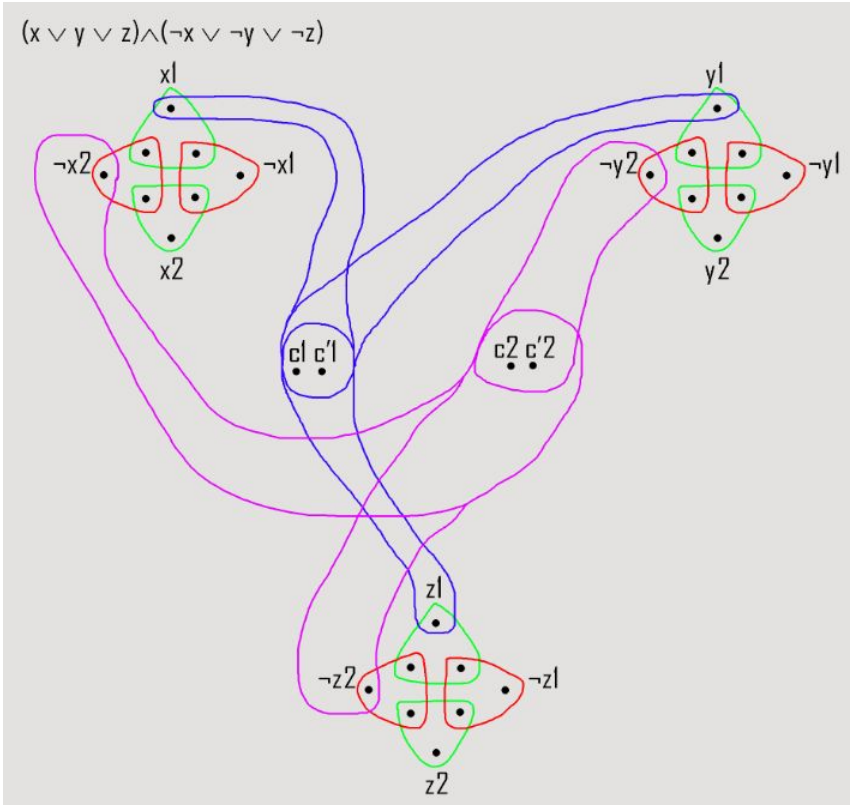
We must choose one hyperedge with one variable for each clause. Because in 3-SAT, it is enough to see that only one variable of each clause is true. After we choose a hyperedge, we must link the negations of chosen variable (if chosen variable is actually a negation, we must link the normal ones.) with its neighbours.



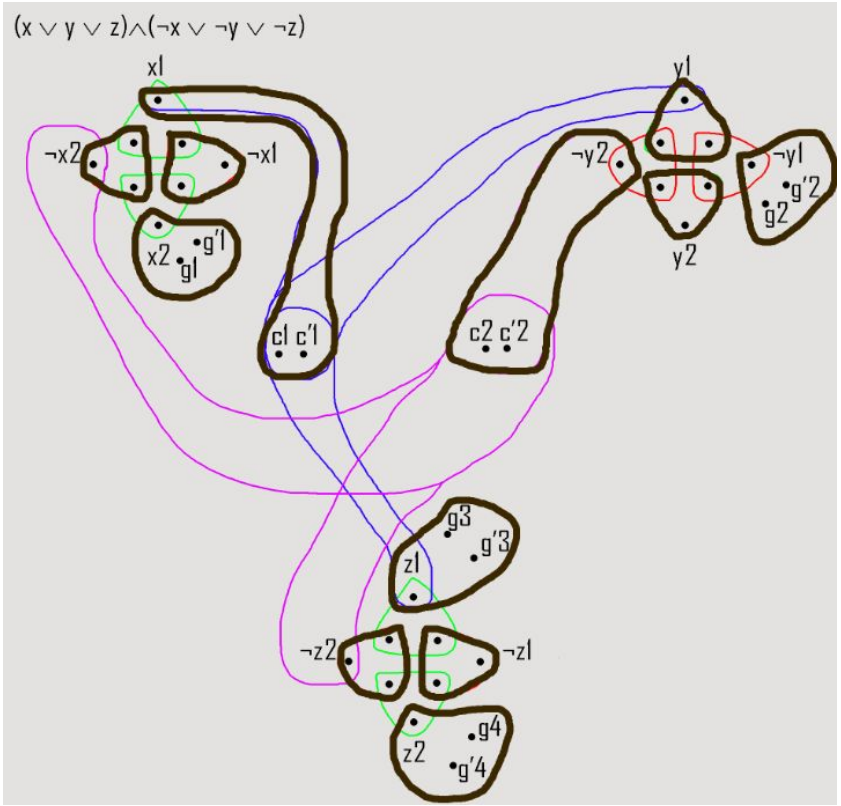
After we do that for all the clauses. we will link last variable points with garbage points (Only variable points.). If there is a free clause, that means our 3-SAT answer is False.

Examples


(x ∨ y ∨ z)∧(¬x ∨ ¬y ∨ ¬z)
x1
¬x2
¬x1
x2
y1
¬y2
¬y1
y2
z1
¬z2
¬z1
z2

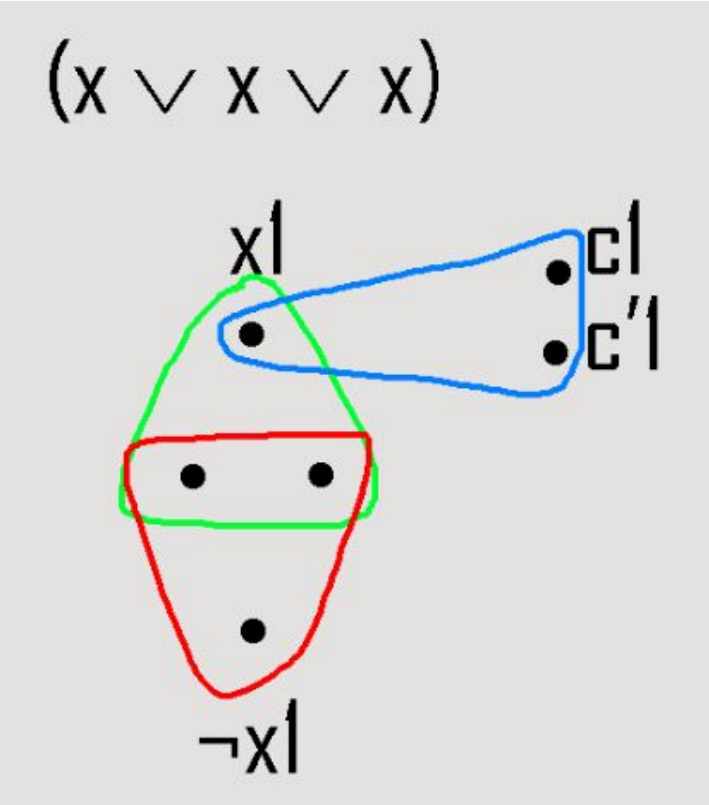

(x ∨ y ∨ z)∧(¬x ∨ ¬y ∨ ¬z)
x1
¬x2
¬x1
x2
y1
¬y2
¬y1
y2
c1 c'1
c2 c'2
z1
¬z2
¬z1
z2

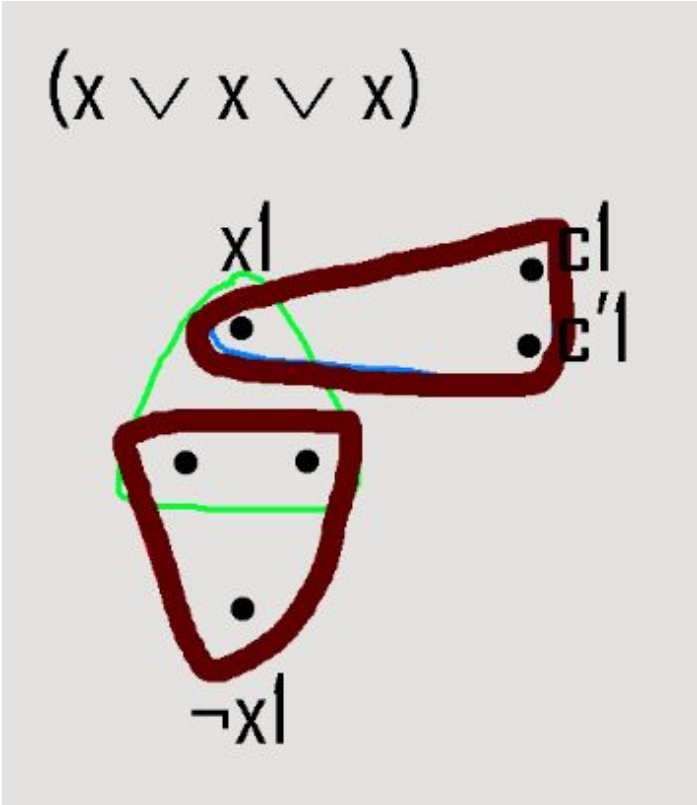
Examples



Since a variable cannot be True and False at the same time. We must choose one of them for each variable.

Examples





Examples